PORTFOLIO PROJECT 04 · JAVA APPLICATION DEVELOPMENT

land-registry-api

A Spring Boot REST API for land parcels and dealing applications – schema under version control, rules that cannot be bypassed, tested from HTTP to database.

Java 17
Spring Boot 3

Java 17 · Spring Boot 3 · Spring Data JPA · Hibernate 6 · Flyway · PostgreSQL · JUnit 5 · Docker · GitHub Actions

Sallehuddin Mohd Zakinuddin · Senior Application Developer · Portfolio project on GitHub

1 Motivation

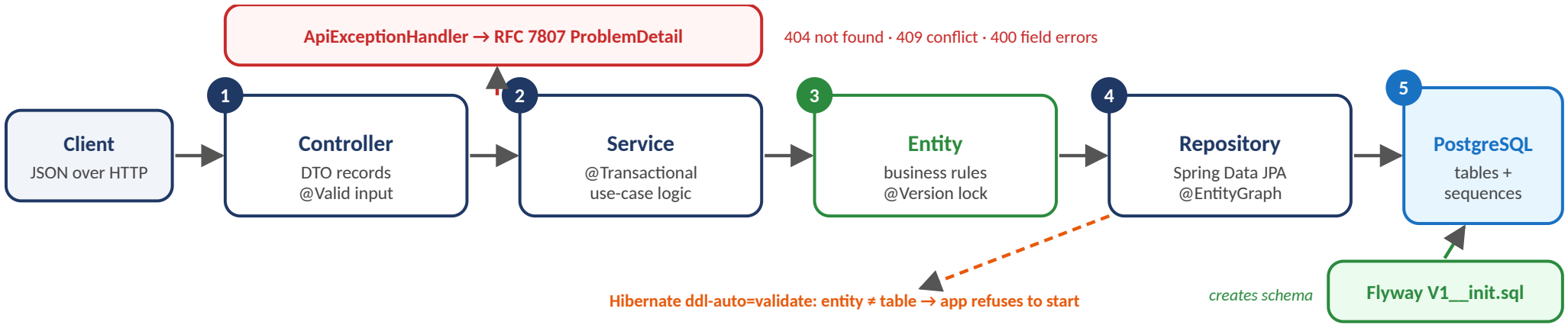
At work I support a large Java / Spring / Hibernate land administration platform from the **operations side**: schema drift, sequence errors, failed releases. Most of those incidents trace back to a handful of application-side habits – schemas changed by hand, entities and tables disagreeing, rules that callers can bypass.

This project is me building the **application side from scratch**, applying the practices that would have prevented the incidents I deal with. *Domain simplified; data fictional.*

2 Objectives

1. A clean, layered **REST API** for parcels and land dealing applications.
2. **Schema under version control** (Flyway), verified against the code at start-up.
3. Business rules – the application workflow – that **cannot be bypassed**.
4. Clear, consistent error responses; input validated against real column limits.
5. Automated tests from HTTP down to the database, run in CI against real PostgreSQL.

3 Layered Architecture



① **Controller.** Thin: maps HTTP to Java records, `@Valid` rejects bad input with field-level errors.

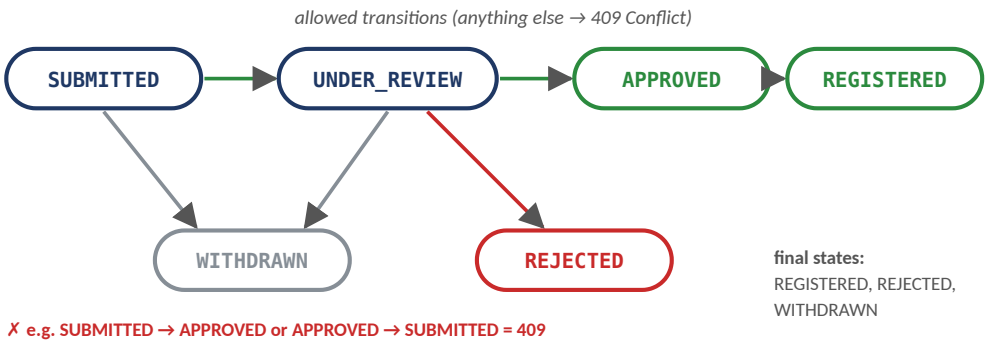
② **Service.** One transaction per use-case; checks like "only one open application per parcel & type".

③ **Entity.** `moveTo()` enforces the workflow; `@Version` stops two officers overwriting each other.

④ **Repository.** Derived queries + `@EntityGraph` to fetch the parcel in the same query (no N+1).

⑤ **Database.** Flyway creates it; CHECK, UNIQUE and FK constraints are the last line of defence.

4 Application Workflow (state machine)



✗ e.g. SUBMITTED → APPROVED or APPROVED → SUBMITTED = 409

Why in the entity, not the controller? Any code path – API, batch job, future admin screen – must go through `LandApplication.moveTo()`, so the rule cannot be skipped by accident.

5 Practices & Why They Matter

Practice	Prevents
`ddl-auto: validate` + Flyway	entity ≠ table (schema drift)
`allocationSize` = `INCREMENT BY`	overlapping ids; 1 DB call per 50 inserts
`@Digits` mirrors `numeric(16,4)`	DB errors becoming HTTP 500s
`@Version` optimistic locking	lost updates between users
`open-in-view:false` + `@EntityGraph`	lazy-loading surprises, N+1 queries
RFC 7807 `ProblemDetail`	inconsistent, unreadable error bodies
Migrations linted by sql-patch-guard	risky DDL reaching the database

6 API in Action

```
POST /api/applications/42/transitions   {"status": "APPROVED"}
409 Conflict   application/problem+json
{ "title": "Conflict", "status": 409,
  "detail": "Cannot move application APP/2026/9F3A1C2B from SUBMITTED to APPROVED;
             allowed: [UNDER_REVIEW, WITHDRAWN]" }

POST /api/parcels   {"lotNo": "", "areaSqm": 12.123456, "tenure": "RENTED"}
400 Bad Request
{ "title": "Validation failed",
  "errors": { "lotNo": "must not be blank", "areaSqm": "numeric value out of bounds",
              "tenure": "must match \"FREEHOLD|LEASEHOLD\"" } }
```

Endpoints: `/api/parcels` · `/api/applications` (filter by status, paged) · `/{id}/transitions` · `/summary` · Swagger UI · `/actuator/health`

7 Testing Strategy

11 unit cases (workflow rules)

7 full-stack API tests

- **Unit:** every allowed / forbidden transition, decision timestamp.
- **Integration:** HTTP → controller → JPA → Flyway-migrated DB; duplicates, 404s, validation, workflow.
- **CI:** boots the real jar on **PostgreSQL 16**; lints migrations with sql-patch-guard.

Status: migration verified on PostgreSQL 16; the Java suite's first run is the GitHub Actions build.

8 Key Concepts Learned

- **The schema is code** – versioned, reviewed, repeatable.
- **Rules live with the data** they protect (rich entity, not a fat controller).
- **Fail fast**: refuse to start rather than corrupt data later.

9 Limitations & Future Work

- No authentication yet → Spring Security with officer roles.
- H2 in tests → Testcontainers PostgreSQL.
- Audit trail table for every status change.

10 Interview Talking Points

Q: Why not `ddl-auto: update`?
It changes production schemas with no review or history.

Q: What does `@Version` do?
Updates check the version; a stale write fails instead of overwriting.

Portfolio project · fictional data · MIT licence